

Имитационное моделирование
Лабораторная работа №3. Агентная модель Daisyworld

Исаева Зарина Исмайлбековна

Содержание

1. Цель работы
2. Задание
3. Теоретическое введение
4. Выполнение лабораторной работы
5. Выводы
6. Список литературы

Список иллюстраций

1. Структура проекта и окружение Julia
2. Модуль модели в VS Code
3. Состояния сетки на шагах 0, 5 и 45
4. Финальный кадр анимации
5. Динамика численности
6. Связанная динамика температуры и светимости
7. Сравнение живого мира с безжизненным контролем
8. Три параметрических исследования
9. Выполненные Jupyter notebooks
10. Автоматические проверки

1. Цель работы

Цель работы — построить агентную модель Daisyworld на клеточной сетке, исследовать обратную связь между популяциями чёрных и белых маргариток, температурой поверхности и внешней светимостью, а также оформить семь вычислительных сценариев в воспроизводимом литературном проекте Julia.

2. Задание

Согласно главе 3 практикума [1] необходимо:

1. создать проект и установить необходимые пакеты;
2. реализовать общий модуль **src/daisyworld.jl**;
3. выполнить базовую визуализацию состояний сетки;
4. записать 60 кадров анимации;
5. рассчитать динамику численности на 1000 шагах;
6. исследовать численность, температуру, альбедо и светимость в сценарии `game`;
7. повторить пространственный, численный и световой опыты для четырёх сочетаний $\text{max_age} \in \{25, 40\}$ и $\text{init_white} \in \{0,2; 0,8\}$;
8. получить из каждого литературного исходника чистый JL, выполненный IPYNB, QMD и HTML;
9. интегрировать полные QMD-фрагменты в отчёт и проверить модель тестами.

3. Теоретическое введение

3.1 Агентный подход

В агентной модели глобальная динамика возникает из локальных правил отдельных агентов. Каждый агент хранит состояние, занимает позицию в среде и изменяется при выполнении шага. `Agents.jl` предоставляет клеточные пространства, планировщик, генератор случайных чисел и операции добавления и удаления агентов [2].

В Daisyworld агентами служат чёрные и белые маргаритки, а средой — периодическая решётка 30×30 . Каждый агент характеризуется видом, возрастом и альбедо. Чёрные растения сильнее нагревают клетку, белые сильнее отражают свет.

3.2 Механизм Daisyworld

Модель Уотсона и Лавлока показывает, как биота может расширить диапазон внешних условий, при которых сохраняется пригодная температура [3]. Она не задаёт глобальный термостат. Саморегуляция возникает из

конкуренции двух видов за свободные клетки и зависимости размножения от локальной температуры. Обзор вариантов Daisyworld показывает, что результат зависит от пространственной структуры, правил смерти, диффузии и скорости изменения светимости [4].

3.3 Температура и размножение

Для клетки с альбедо A и светимостью L используется поглощённое излучение $Q = (1 - A)L$. Целевая локальная температура вычисляется как

$$T_{local} = 72 \ln Q + 80, Q > 0.$$

После локального нагрева половина температурного поля диффундирует по восьми соседям. Вероятность прорастания семени равна

$$p(T) = \max(0, \min(1, 0,1457 T - 0,0032 T^2 - 0,6443)).$$

Агент стареет на каждом шаге и удаляется при достижении максимального возраста. Освободившуюся клетку может занять семя соседнего растения.

4. Выполнение лабораторной работы

4.1 Окружение и структура проекта

Проект разделён на модуль модели, общие средства экспериментов, семь литературных сценариев, тесты, таблицы, графики и производные документы. DrWatson задаёт устойчивые пути к данным и результатам [5].

```

rhktrlwmd@Mac % pwd
/workspace/labs/lab03/project
rhktrlwmd@Mac % find . -maxdepth 2 -type d | sort
.
./data
./data/daisyworld
./data/daisyworld-animate
./data/daisyworld-count
./data/daisyworld-count__param
./data/daisyworld-luminosity
./data/daisyworld-luminosity__param
./data/daisyworld__param
./markdown
./notebooks
./notebooks/daisyworld
./notebooks/daisyworld-animate
./notebooks/daisyworld-count
./notebooks/daisyworld-count__param
./notebooks/daisyworld-luminosity
./notebooks/daisyworld-luminosity__param
./notebooks/daisyworld__param
./plots
./scripts
./scripts/daisyworld
./scripts/daisyworld-animate
./scripts/daisyworld-count
./scripts/daisyworld-count__param
./scripts/daisyworld-luminosity
./scripts/daisyworld-luminosity__param
./scripts/daisyworld__param
./src
./test
rhktrlwmd@Mac %

```

Рисунок 1: Структура проекта Daisyworld

```

rhktrlwmd@Mac % pwd
/workspace/labs/lab03/project
rhktrlwmd@Mac % julia --version
julia version 1.11.9
rhktrlwmd@Mac % julia --project=. -e 'using Pkg; Pkg.status()'
Project DaisyworldLab v3.0.0
Status `~/workspace/labs/lab03/project/Project.toml`
 [46ada45e] Agents v7.0.3
 [336ed68f] CSV v0.10.17
 [13f3f980] CairoMakie v0.15.14
 [a93c6f00] DataFrames v1.8.2
 [634d3b9d] DrWatson v2.19.1
 [7073ff75] IJulia v1.34.4
 [033835bb] JLD2 v0.6.6
 [98b081ad] Literate v2.21.0
 [ee78f7c6] Makie v0.24.14
 [91a5bcdd] Plots v1.41.7
 [10745b16] Statistics v1.11.5
 [2913bbd2] StatsBase v0.34.13
 [9a3f8284] Random v1.11.0
rhktrlwmd@Mac %

```

Рисунок 2: Закреплённое окружение Julia

name	=	"DaisyworldLab"
uuid	=	"67574b67-7156-42e8-81c1-d1b570a65721"
authors	=	["rhktrlwmd"]
version	=	"3.0.0"
[deps]		
Agents	=	"46ada45e-f475-11e8-01d0-f70cc89e6671"
CSV	=	"336ed68f-0bac-5ca0-87d4-7b16caf5d00b"

```

CairoMakie          = "13f3f980-e62b-5c42-98c6-ff1f3baf88f0"
DataFrames          = "a93c6f00-e57d-5684-b7b6-d8193f3e46c0"
DrWatson            = "634d3b9d-ee7a-5ddf-bec9-22491ea816e1"
IJulia              = "7073ff75-c697-5162-941a-fcdaad2a7d2a"
JLD2                = "033835bb-8acc-5ee8-8aae-3f567f8a3819"
Literate            = "98b081ad-f1c9-55d3-8b20-4c87d4299306"
Makie               = "ee78f7c6-11fb-53f2-987a-cfe4a2b5a57a"
Plots               = "91a5bcd-d55d7-5caf-9e0b-520d859cae80"
Random              = "9a3f8284-a2c9-5f02-9a11-845980a1fd5c"
Statistics           = "10745b16-79ce-11e8-11f9-7d13ad32a3b2"
StatsBase           = "2913bbd2-ae8a-5f71-8c99-4fb6c76f3a91"

[compat]
Agents              = "7.0.3"
CairoMakie          = "0.15.13"
JLD2                = "0.6.6"
Makie               = "0.24.13"
StatsBase           = "0.34.13"
julia               = "1.11"

[preferences.Makie]
precompile_workload = false

[preferences.CairoMakie]
precompile_workload = false

```

Листинг 1 — полный файл **project/Project.toml**.

```

SOURCES = scripts/daisyworld.jl scripts/daisyworld-animate.jl scripts/daisyworld-count.jl scripts/daisyworld-
luminosity.jl scripts/daisyworld__param.jl scripts/daisyworld-count__param.jl scripts/daisyworld-luminosity__param.jl

.PHONY: setup tangle run notebooks docs test verify all clean-results

setup:
    julia --project=. scripts/setup_environment.jl

tangle:
    julia --project=. scripts/tangle.jl $(SOURCES)

run:
    @for source in $(SOURCES); do julia --project=. $$source || exit $$?; done

notebooks:
    julia --project=. scripts/tangle.jl --execute $(SOURCES)

docs:
    @for source in $(SOURCES); do \
        name=$(basename $$source .jl); \
        (cd notebooks/$$name && quarto render $$name.ipynb --to html --embed-resources --no-execute --output $$name.html) || \
    exit $$?; \
    mv notebooks/$$name/$$name.html markdown/$$name/$$name.html; \
done

test:
    julia --project=. test/runtests.jl

verify:
    @clean_scripts=$(find scripts -mindepth 2 -maxdepth 2 -name '*.jl' | wc -l | tr -d ' '); \

```

```

notebooks=$(find notebooks -mindepth 2 -maxdepth 2 -name '*.ipynb' | wc -l | tr -d ' '); \
qmd=$(find markdown -mindepth 2 -maxdepth 2 -name '*.qmd' | wc -l | tr -d ' '); \
html=$(find markdown -mindepth 2 -maxdepth 2 -name '*.html' | wc -l | tr -d ' '); \
figures=$(find plots -type f \( -name '*.png' -o -name '*.mp4' \) | wc -l | tr -d ' '); \
printf 'clean Julia scripts: %s\nexecuted notebooks: %s\nQuarto documents: %s\nHTML documents: %s\nresult figures and\nanimation: %s\n' \
      "$$clean_scripts" "$$notebooks" "$$qmd" "$$html" "$$figures"; \
test "$$clean_scripts" = 7; test "$$notebooks" = 7; test "$$qmd" = 14; test "$$html" = 7; test "$$figures" = 22
all:                                run                                notebooks                                docs                                test                                verify

clean-results:
rm -rf data plots notebooks markdown

```

Листинг 2 — полный файл **project/Makefile**.

```

using                                                                                               Pkg

Pkg.instantiate()
Pkg.precompile()

using                                                                                               IJulia
project_path                                             =                               dirname(Base.active_project())
IJulia.installkernel(
                                "Julia"                               Lab03                               1.11",
                                "--project=$(project_path)";
                                env=Dict("JULIA_DEPOT_PATH"=>join(DEPOT_PATH,
                                ':')),
)

```

Листинг 3 — настройка окружения и ядра Jupyter.

4.2 Общий модуль модели

Модуль проверяет входные параметры, создаёт периодическую сетку, размещает агентов без пересечений и выполняет локальный нагрев, синхронную диффузию, распространение семян и изменение светимости. Копия температурного поля исключает зависимость диффузии от порядка обхода клеток.


```

A daisyworld@ src
22 function update_surface_temperature!(pos, model)
23 end
24
25 function diffuse_temperature!(model)
26     before = copy(model.temperature)
27     for pos in positions(model)
28         neighbours = collect(nearby_positions(pos, model))
29         neighbour_mean = mean(before[p...] for p in neighbours)
30         model.temperature[pos...] =
31             (1 - model.diffusion_ratio) * before[pos...] +
32             model.diffusion_ratio * neighbour_mean
33     end
34     return nothing
35 end
36
37 function propagate!(pos, model)
38     isempty(pos, model) && return nothing
39     parent = model[id_in_position(pos, model)]
40     temperature = model.temperature[pos...]
41     probability = clamp(
42         0.1457 * temperature - 0.0032 * temperature^2 - 0.6443,
43         0.0,
44         1.0,
45     )
46     rand(abmrng(model)) < probability || return nothing
47     target = random_nearby_position(pos, model, 1, p -> isempty(p, model))
48     isnothing(target) || add_agent!(target, model, parent.breed, 0, parent.albedo)
49     return nothing
50 end
51
52 function daisy_step!(agent::Daisy, model)
53     agent.age += 1
54     agent.age >= model.max_age && remove_agent!(agent, model)
55     return nothing
56 end
57
58 function solar_activity!(model)
59     if model.scenario == :ramp
60         200 < model.tick <= 400 && (model.solar_luminosity += model.solar_change)
61         500 < model.tick <= 750 && (model.solar_luminosity -= model.solar_change / 2)
62     elseif model.scenario == :change
63         model.solar_luminosity += model.solar_change
64     end
65 end

```

Рисунок 3: Общий модуль Daisyworld в VS Code

module	Daisyworld
using Agents	
using Random	
using Statistics	
using StatsBase: sample	
export Daisy, create_world, advance!, observe, collect_history,	
update_surface_temperature!, diffuse_temperature!, propagate!, solar_activity!	
@agent struct Daisy(GridAgent{2})	
breed::Symbol	
age::Int	
albedo::Float64	
end	
function local_heating(albedo, luminosity)	
absorbed = (1 - albedo) * luminosity	
return absorbed > 0 ? 72 * log(absorbed) + 80 : 80.0	
end	
function update_surface_temperature!(pos, model)	
albedo = isempty(pos, model) ? model.surface_albedo :	
model[id_in_position(pos, model)].albedo	
target = local_heating(albedo, model.solar_luminosity)	
model.temperature[pos...] = (model.temperature[pos...] + target) / 2	
return nothing	
end	
function diffuse_temperature!(model)	
before = copy(model.temperature)	
for pos in positions(model)	
neighbours = collect(nearby_positions(pos, model))	
neighbour_mean = mean(before[p...] for p in neighbours)	

```

        model.temperature[pos...] =
            (1 - model.diffusion_ratio) * before[pos...] +
            model.diffusion_ratio * neighbour_mean
    end
    return nothing
end

function propagate!(pos, model)
    isempty(pos, model) && return nothing
    parent = model[id_in_position(pos, model)]
    temperature = model.temperature[pos...]
    probability = clamp(
        0.1457 * temperature - 0.0032 * temperature^2 - 0.6443,
        0.0,
        1.0,
    )
    rand(abmrng(model)) < probability || return nothing
    target = random_nearby_position(pos, model, 1, p -> isempty(p, model))
    isnothing(target) || add_agent!(target, model, parent.breed, 0, parent.albedo)
    return nothing
end

function daisy_step!(agent::Daisy, model)
    agent.age >= model.max_age && remove_agent!(agent, model)
    agent.age += 1
    return nothing
end

function solar_activity!(model)
    if model.scenario == :ramp
        200 < model.tick <= 400 && (model.solar_luminosity += model.solar_change)
        500 < model.tick <= 750 && (model.solar_luminosity -= model.solar_change / 2)
    elseif model.scenario == :change
        model.solar_luminosity += model.solar_change
    end
    return nothing
end

function world_step!(model)
    for pos in positions(model)
        update_surface_temperature!(pos, model)
    end
    for pos in positions(model)
        diffuse_temperature!(model)
        propagate!(pos, model)
    end
    model.tick += 1
    solar_activity!(model)
    return nothing
end

function create_world(;
    griddims=(30, 30),
    init_white=0.2,
    init_black=0.2,
    albedo_white=0.75,
    albedo_black=0.25,
    surface_albedo=0.4,
    solar_change=0.005,
    solar_luminosity=1.0,
    diffusion_ratio=0.5,
    scenario=:default,
    seed=165,
)
    all(>=(3), griddims) || throw(ArgumentError("grid dimensions must be at least 3"))

```

```

        max_age > 0 || throw(ArgumentError("max_age must be positive"))
        0 <= init_white <= 1 || throw(ArgumentError("init_white outside [0, 1]"))
        0 <= init_black <= 1 || throw(ArgumentError("init_black outside [0, 1]"))
        init_white + init_black <= 1 || throw(ArgumentError("initial cover exceeds the grid"))
        all(x -> 0 <= x <= 1, (albedo_white, albedo_black, surface_albedo)) ||
            throw(ArgumentError("albedo outside [0, 1]"))
        0 <= diffusion_ratio <= 1 || throw(ArgumentError("diffusion_ratio outside [0, 1]"))
        scenario in (:default, :ramp, :change) || throw(ArgumentError("unknown scenario"))

        properties = Dict{Symbol,Any}()
        :max_age=>max_age, :surface_albedo=>surface_albedo,
        :solar_luminosity=>solar_luminosity, :solar_change=>solar_change,
        :diffusion_ratio=>diffusion_ratio, :scenario=>scenario,
        :tick=>0, :temperature=>zeros(griddims),
    )

    model = StandardABM(
        Daisy, GridSpaceSingle(griddims; periodic=true);
        properties, rng=MersenneTwister(seed),
        agent_step! = daisy_step!, model_step! = world_step!,
        scheduler=Schedulers.fastest,
    )

    cells = collect(positions(model))
    white_cells = sample(abmrng(model), cells, floor(Int, init_white * length(cells)); replace=false)
    for pos in white_cells
        add_agent!(pos, model, :white, rand(abmrng(model), 0:max_age-1), albedo_white)
    end
    free_cells = setdiff(cells, white_cells)
    black_cells = sample(abmrng(model), free_cells, floor(Int, init_black * length(cells)); replace=false)
    for pos in black_cells
        add_agent!(pos, model, :black, rand(abmrng(model), 0:max_age-1), albedo_black)
    end
    for pos in positions(model)
        update_surface_temperature!(pos, model)
    end
    return model
end

advance!(model, steps=1) = Agents.step!(model, steps)

function
    black = count(a -> a.breed == :black, allagents(model))
    white = count(a -> a.breed == :white, allagents(model))
    cell_albedo = mean(
        isempty(pos, model) ? model.surface_albedo : model[id_in_position(pos, model)].albedo
        for pos in positions(model)
    )
    mean_temperature = mean(model.temperature)
    return (
        time=model.tick, black, white, total=black + white,
        empty=length(model.temperature) - black - white,
        temperature=mean_temperature,
        luminosity=model.solar_luminosity,
        albedo=cell_albedo,
        regulation_error=abs(mean_temperature - 22.5),
    )
end

function
    collect_history(model, steps)
    rows = [observe(model)]
end

```

```

                                for _ in 1:steps
                                    advance!(model)
                                    push!(rows, observe(model))
                                end
                                return rows
end

end

```

Листинг 4 — полный модуль `project/src/daisyworld.jl`.

```

using Agents
using CairoMakie
using CSV
using DataFrames
using DrWatson
using Statistics

include(joinpath(@__DIR__, "daisyworld.jl"))
using .Daisyworld

set_theme!(Theme(font="DejaVu Sans", fontsize=16))

scenario_data(name, file) = (mkpath(datadir(name)); datadir(name, file))
scenario_plot(name, file) = (mkpath(plotsdir(name)); plotsdir(name, file))
report_figure(file) = (mkpath(projectdir("../image", "results"); projectdir("../image", "results", file))

function save_figure(fig, scenario, filename; report_name=nothing)
    save(scenario_plot(scenario, filename), fig)
    isnnothing(report_name) || save(report_figure(report_name), fig)
    return fig
end

function snapshot_figure(model; title="Пространственное состояние Daisyworld")
    fig = Figure(size=(880, 710))
    ax = Axis(fig[1, 1], title="$title, t=$(model.tick)", xlabel="x", ylabel="y", aspect=1)
    heat = heatmap!(ax, model.temperature; colormap=:thermal, colorrange=(-20, 60))
    for (breed, colour, stroke) in ((:black, :black, :white), (:white, :white, :black))
        points = [Point2f(a.pos...) for a in allagents(model) if a.breed == breed]
        isempty(points) || scatter!(ax, points; color=colour, markersize=8, strokecolor=stroke, strokewidth=0.5)
    end
    Colorbar(fig[1, 2], heat; label="Температура, °C")
    return fig
end

function count_figure(frame; title="Динамика численности маргариток")
    fig = Figure(size=(1000, 560))
    ax = Axis(fig[1, 1], title=title, xlabel="Модельный шаг", ylabel="Число растений")
    lines!(ax, frame.time, frame.black; color=:black, linewidth=2, label="Чёрные")
    lines!(ax, frame.time, frame.white; color=:orange, linewidth=2, label="Белые")
    axislegend(ax; position=:rt)
    return fig
end

function dynamics_figure(frame; title="Обратная связь живой и неживой среды")

```

```

fig = Figure(size=(1000, 960))
ax1 = Axis(fig[1, 1], title=title, ylabel="Число растений")
ax2 = Axis(fig[2, 1], ylabel="Температура, °C")
ax3 = Axis(fig[3, 1], ylabel="Светимость")
ax4 = Axis(fig[4, 1], ylabel="Альбедо", xlabel="Модельный шаг")
lines!(ax1, frame.time, frame.black; color=:black, label="Чёрные")
lines!(ax1, frame.time, frame.white; color=:orange, label="Белые")
axislegend(ax1; position=:rt)
lines!(ax2, frame.time, frame.temperature; color=:firebrick)
lines!(ax3, frame.time, frame.luminosity; color=:royalblue)
lines!(ax4, frame.time, frame.albedo; color=:darkgreen)
linkxaxes!(ax1, ax2, ax3, ax4)
hidexdecorations!(ax1; grid=false)
hidexdecorations!(ax2; grid=false)
hidexdecorations!(ax3; grid=false)
return fig
end

parameter_sets() = dict_list(Dict{
    :griddims=>(30, 30), :max_age=>[25, 40],
    :init_white=>[0.2, 0.8], :init_black=>0.2,
    :albedo_white=>0.75, :albedo_black=>0.25,
    :surface_albedo=>0.4, :solar_change=>0.005,
    :solar_luminosity=>1.0, :scenario=>:default, :seed=>165,
})

function history_frame(model, steps)
    return DataFrame(collect_history(model, steps))
end

function save_history(frame, scenario; filename="trajectory.csv")
    CSV.write(scenario_data(scenario, filename), frame)
    return frame
end

function extinction_time(frame)
    index = findfirst(==(0), frame.total)
    return isnothing(index) ? missing : frame.time[index]
end

```

Листинг 5 — общие средства экспериментов и визуализации.

4.3 Базовая пространственная визуализация

Базовые параметры: сетка 30×30 , возраст 25, начальные доли обоих видов 0,2, альбедо белых 0,75, чёрных 0,25, почвы 0,4, светимость 1 и seed 165.

4.3.1 *Daisyworld*: пространственная эволюция

Базовый опыт воспроизводит состояния сетки 30×30 на шагах 0, 5 и 45. Чёрные маргаритки поглощают больше излучения, белые отражают его.

4.3.1.1 Подготовка модели

```
using                                     DrWatson
@quickactivate                           "DaisyworldLab"
include(scrdir("experiment_tools.jl"))
```

4.3.1.2 Снимки заданных моментов

```
model = create_world(seed=165)
rows = NamedTuple{(), Tuple{}}()
for target in (0, 5, 45)
    advance!(model, target)
    fig = push!(rows, target)
    model.tick()
    observe(model)
    snapshot_figure(model)
    save_figure(
        fig, "daisyworld", "step-$(lpad(target, 3, '0')).png";
        report_name="baseline-step-$(lpad(target, 3, '0')).png",
    )
    display(fig)
end
```

4.3.1.3 Таблица наблюдений

```
summary = DataFrame{rows}()
CSV.write(scenario_data("daisyworld", summary), "snapshots.csv"),
```

Листинг 6 — литературный сценарий `scripts/daisyworld.jl`.

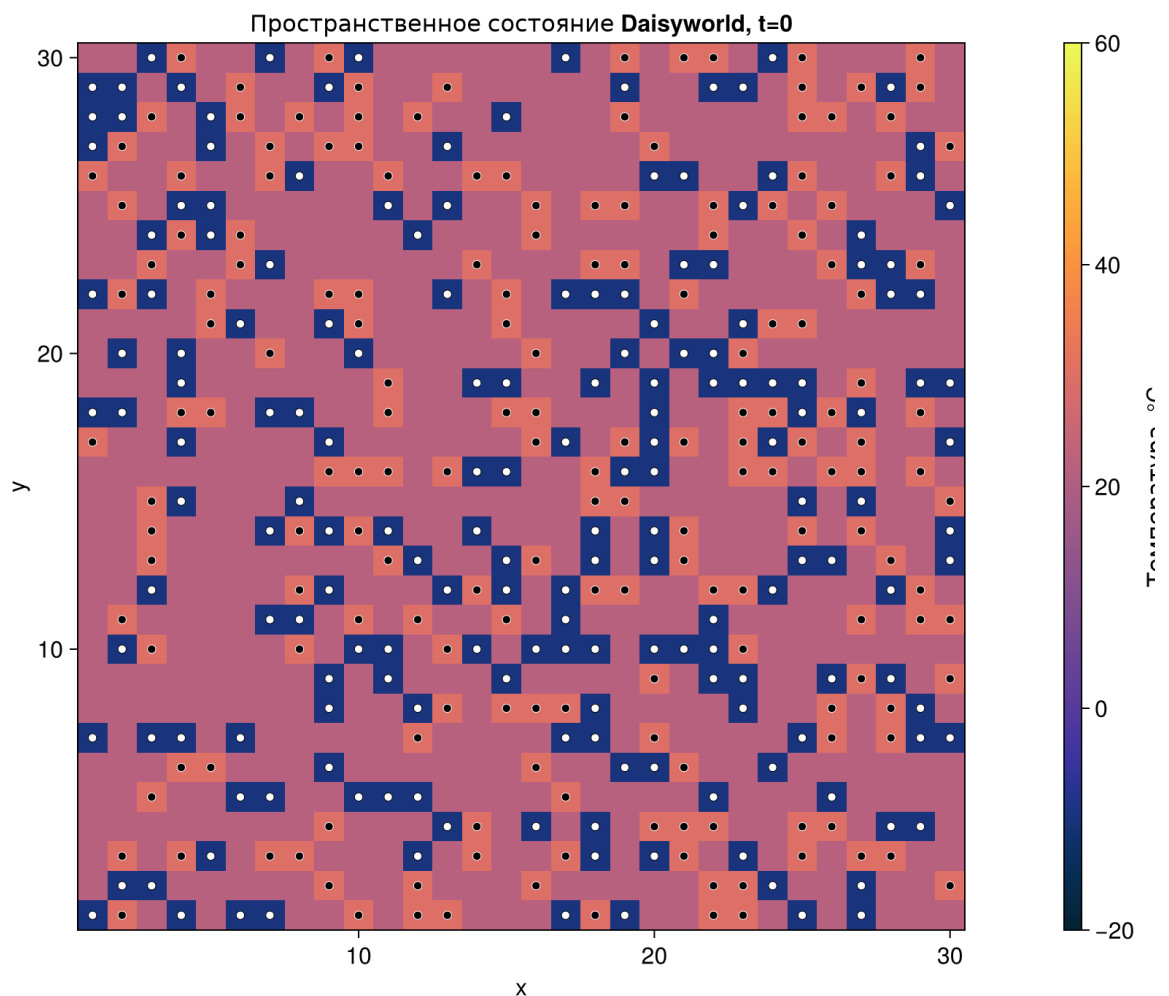


Рисунок 4: Шаг 0

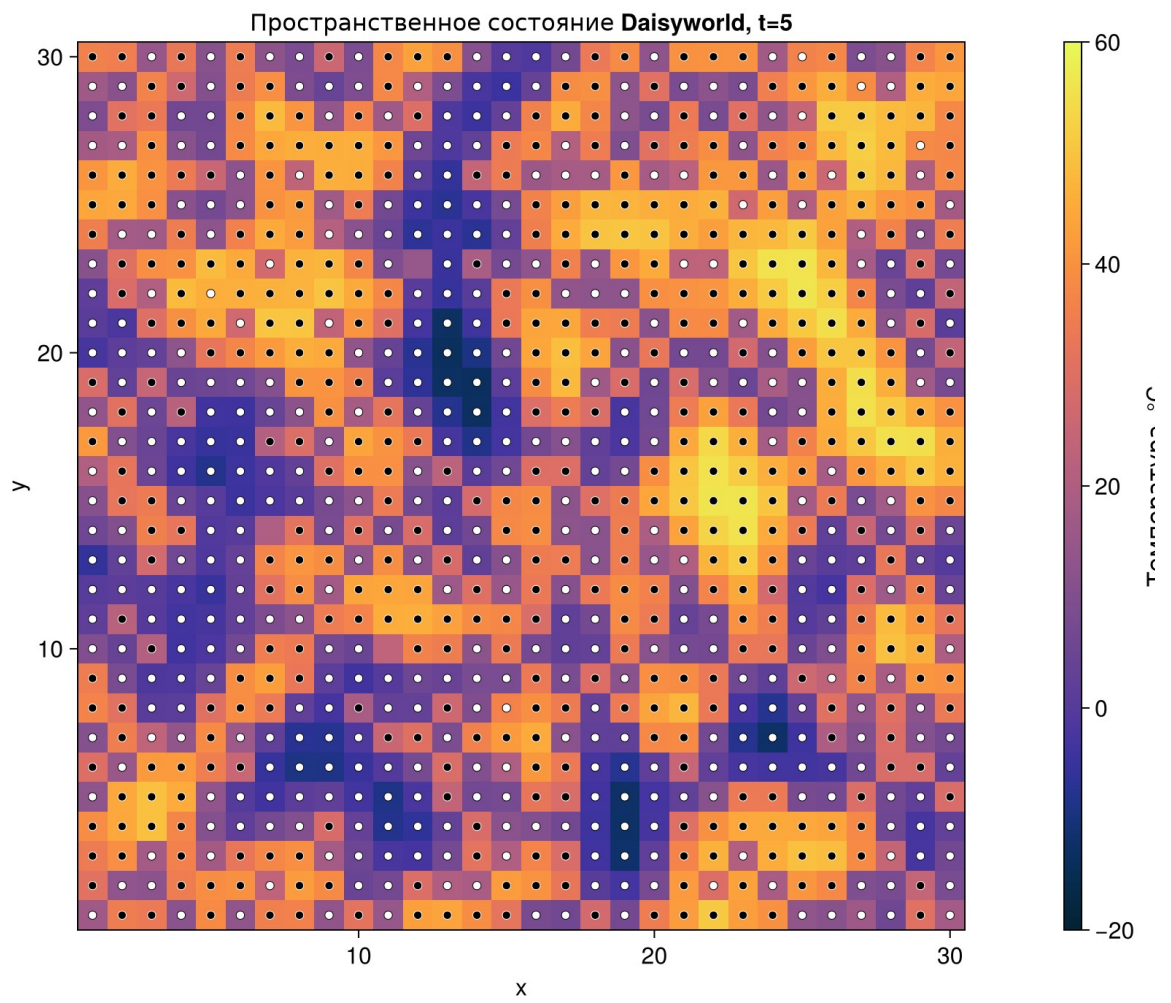


Рисунок 5: Шаг 5

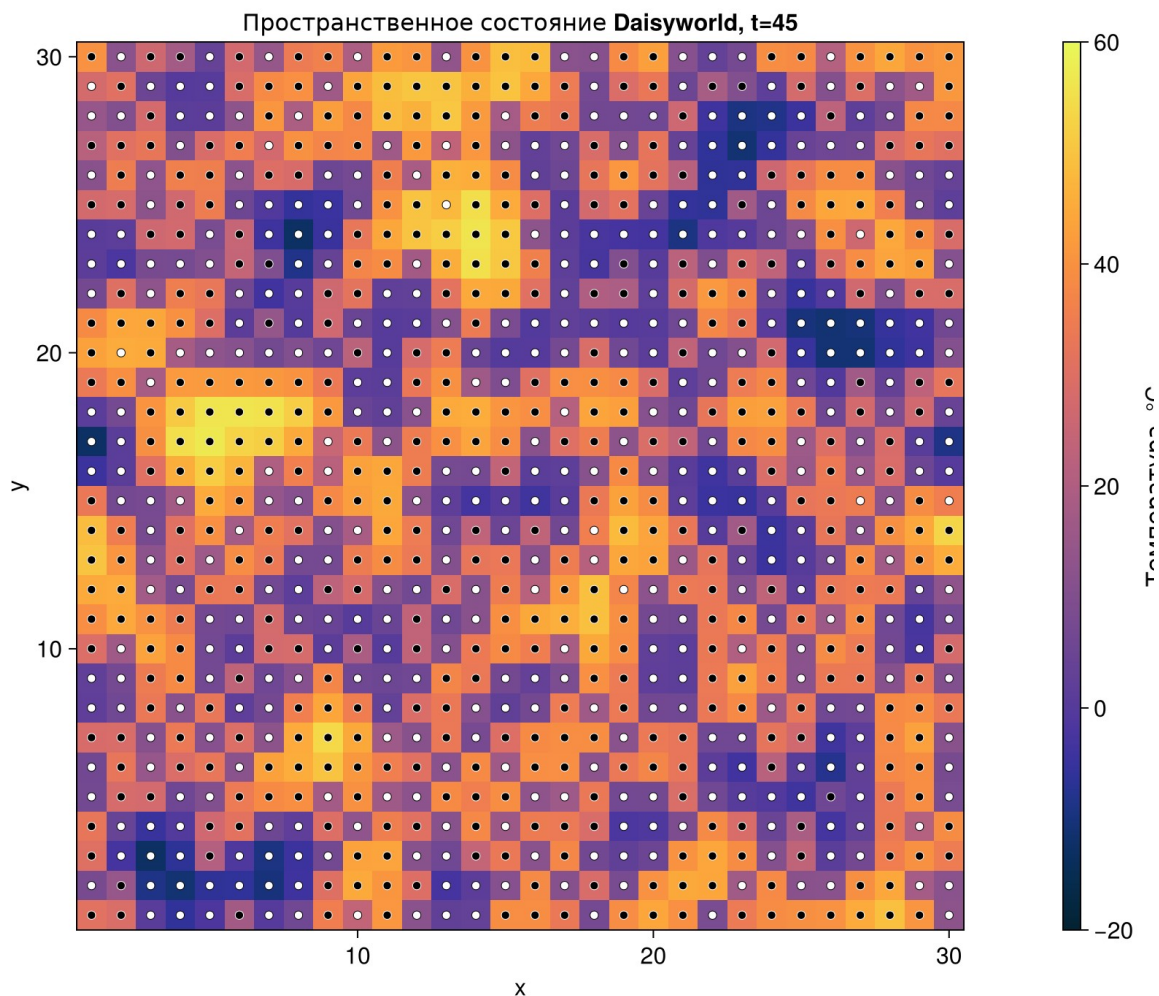


Рисунок 6: Шаг 45

На шаге 0 имеется по 180 растений каждого вида, средняя температура равна 16,91 °C. К шагу 5 занято 897 клеток, температура повышается до 23,03 °C. На шаге 45 сетка полностью занята: 469 чёрных и 431 белая маргаритка, температура составляет 21,70 °C.

4.4 Анимация модели

Отдельный сценарий сохраняет 60 последовательных состояний в MP4 с частотой 10 кадров/с. Вместе с видео записывается CSV с наблюдаемыми величинами.

4.4.1 Daisyworld: анимация 60 состояний

Анимация показывает первые 60 шагов базового опыта при фиксированном seed.

4.4.1.1 Подготовка модели и сцены


```
include(sourcedir("experiment_tools.jl"))

model = create_world(seed=165)
fig = Figure(size=(880, 710))
ax = Axis(fig[1, 1], title="Daisyworld", xlabel="x", ylabel="y", aspect=1)
temperatures = Observable(copy(model.temperature))
black_points = Observable(Point2f[])
white_points = Observable(Point2f[])
heat = heatmap!(ax, temperatures; colormap=:thermal, colrange=(-20, 60))
scatter!(ax, black_points; color=:black, markersize=8)
scatter!(ax, white_points; color=:white, strokecolor=:black, strokewidth=0.5, markersize=8)
Colorbar(fig[1, 2], heat; label="Температура, °C")
```

4.4.1.2 Запись последовательности

```
rows = NamedTuple[]
record(fig, scenario_plot("daisyworld-animate", "daisyworld-evolution.mp4"), 0:59; framerate=10) do frame
    frame > 0 && advance!(model)
    temperatures[] = copy(model.temperature)
    black_points[] = [Point2f(a.pos...) for a in allagents(model) if a.breed == :black]
    white_points[] = [Point2f(a.pos...) for a in allagents(model) if a.breed == :white]
    ax.title = "Daisyworld, t=$(model.tick)"
    push!(rows, observe(model))
end
```

4.4.1.3 Данные кадров и финальное состояние

```
frame_data = DataFrame{rows}
CSV.write(scenario_data("daisyworld-animate", "frames.csv"), frame_data)
save(report_figure("animation-final-state.png"), fig)
display(fig)
frame_data[1:10:end, :]
```

Листинг 7 — сценарий `scripts/daisyworld-animate.jl`.

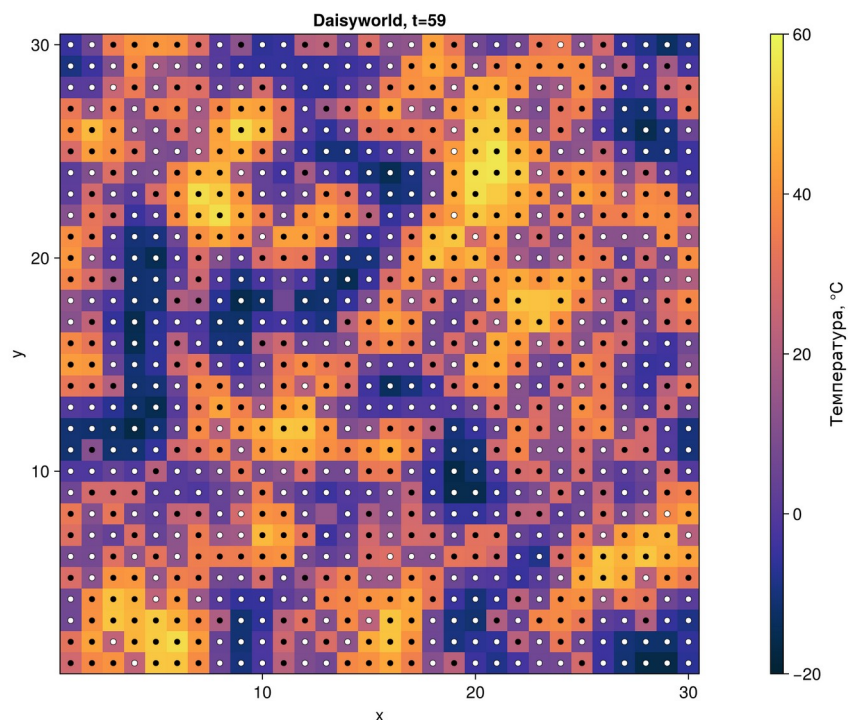


Рисунок 7: Финальное состояние 60-кадровой анимации

4.5 Динамика численности

4.5.1 Daisyworld: динамика численности

На горизонте 1000 шагов прослеживаются обе популяции и момент их исчезновения.

4.5.1.1 Расчёт базового опыта

```
using DrWatson
@quickactivate "DaisyworldLab"
include(scrdir("experiment_tools.jl"))

model = create_world(solar_luminosity=1.0, seed=165)
trajectory = save_history(history_frame(model, 1000), "daisyworld-count")
```

4.5.1.2 График и сводка

```
fig = count_figure(trajectory)
save_figure(fig, "daisyworld-count", "population-dynamics.png";
report_name="baseline-population-dynamics.png")

display(fig)

summary = DataFrame(((
    final_black=last(trajectory.black),
    final_white=last(trajectory.white),
    peak_black=maximum(trajectory.black),
    peak_white=maximum(trajectory.white),
    extinction_step=extinction_time(trajectory),
)))
CSV.write(scenario_data("daisyworld-count", "summary.csv"), summary)
```

Листинг 8 — сценарий `scripts/daisyworld-count.jl`.

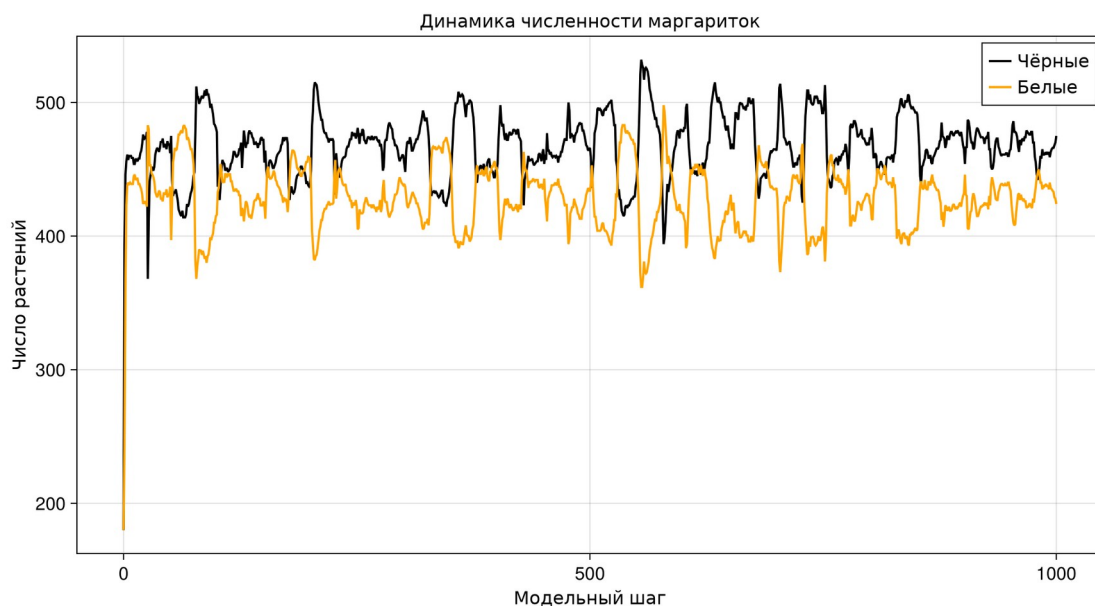
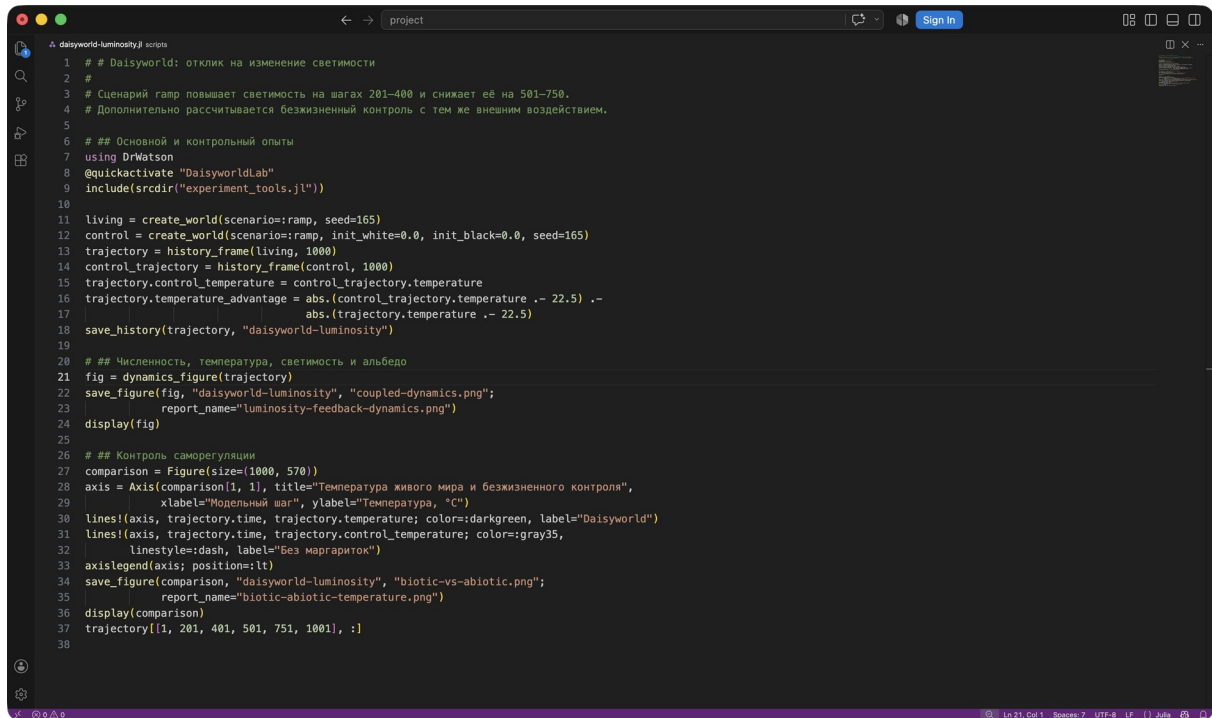


Рисунок 8: Динамика численности в базовом опыте

За 1000 шагов оба вида сохраняются. Конечное состояние содержит 475 чёрных и 424 белые маргаритки; максимумы равны 532 и 498. Полного вымирания нет.

4.6 Изменение светимости

В сценарии `ramp` светимость возрастает от 1 до 2 на шагах 201–400, сохраняется до шага 500 и затем уменьшается до 1,375 на шагах 501–750.



```
1 # Daisyworld: отклик на изменение светимости
2 #
3 # Сценарий ramp повышает светимость на шагах 201–400 и снижает её на 501–750.
4 # Дополнительно рассчитывается безжизненный контроль с тем же внешним воздействием.
5
6 ## Основной и контрольный опыты
7 using DrWatson
8 @quickactivate "DaisyworldLab"
9 include(scrdir("experiment_tools.jl"))
10
11 living = create_world(scenario=:ramp, seed=165)
12 control = create_world(scenario=:ramp, init_white=0.0, init_black=0.0, seed=165)
13 trajectory = history_frame(living, 1000)
14 control_trajectory = history_frame(control, 1000)
15 trajectory.control_temperature = control_trajectory.temperature
16 trajectory.temperature_advantage = abs.(control_trajectory.temperature .- 22.5) .-
17                                     abs.(trajectory.temperature .- 22.5)
18 save_history(trajectory, "daisyworld-luminosity")
19
20 ## Численность, температура, светимость и альbedo
21 fig = dynamics_figure(trajectory)
22 save_figure(fig, "daisyworld-luminosity", "coupled-dynamics.png";
23             report_name="luminosity-feedback-dynamics.png")
24 display(fig)
25
26 ## Контроль саморегуляции
27 comparison = Figure(size=(1000, 570))
28 axis = Axis(comparison[1, 1], title="Температура живого мира и безжизненного контроля",
29             xlabel="Модельный шаг", ylabel="Температура, °C")
30 lines!(axis, trajectory.time, trajectory.temperature; color=:darkgreen, label="Daisyworld")
31 lines!(axis, trajectory.time, trajectory.control_temperature; color=:gray35,
32        linestyle=:dash, label="Без маргариток")
33 axislegend(axis, position=:lt)
34 save_figure(comparison, "daisyworld-luminosity", "biotic-vs-abiotic.png";
35             report_name="biotic-abiotic-temperature.png")
36 display(comparison)
37 trajectory[[1, 201, 401, 501, 751, 1001], :]
38
```

Рисунок 9: Сценарий изменения светимости в VS Code

4.6.1 Daisyworld: отклик на изменение светимости

Сценарий `ramp` повышает светимость на шагах 201–400 и снижает её на 501–750. Дополнительно рассчитывается безжизненный контроль с тем же внешним воздействием.

4.6.1.1 Основной и контрольный опыты

<code>using</code>	DrWatson
<code>@quickactivate</code>	"DaisyworldLab"
<code>include(scrdir("experiment_tools.jl"))</code>	
<code>living</code>	<code>= create_world(scenario=:ramp, seed=165)</code>
<code>control</code>	<code>= create_world(scenario=:ramp, init_white=0.0, init_black=0.0, seed=165)</code>
<code>trajectory</code>	<code>= history_frame(living, 1000)</code>
<code>control_trajectory</code>	<code>= history_frame(control, 1000)</code>
<code>trajectory.control_temperature</code>	<code>= control_trajectory.temperature</code>
<code>trajectory.temperature_advantage</code>	<code>= abs.(control_trajectory.temperature .- 22.5) .-</code> <code>abs.(trajectory.temperature .- 22.5)</code>
<code>save_history(trajectory, "daisyworld-luminosity")</code>	

4.6.1.2 Численность, температура, светимость и альbedo

```
fig = dynamics_figure(trjectory)
save_figure(fig, "daisyworld-luminosity", "coupled-dynamics.png";
report_name="luminosity-feedback-dynamics.png")
display(fig)
```

4.6.1.3 Контроль саморегуляции

```
comparison = Figure(size=(1000, 570))
axis = Axis(comparison[1, 1], title="Температура живого мира и безжизненного контроля",
xlabel="Модельный шаг", ylabel="Температура, °C")
lines!(axis, trjectory.time, trjectory.temperature; color=:darkgreen, label="Daisyworld")
lines!(axis, trjectory.time, trjectory.control_temperature; color=:gray35,
linestyle=:dash, label="Без маргариток")
axislegend(axis; position=:lt)
save_figure(comparison, "daisyworld-luminosity", "biotic-vs-abiotic.png";
report_name="biotic-abiotic-temperature.png")
display(comparison)
trjectory[[1, 201, 401, 501, 751, 1001], :]
```

Листинг 9 — сценарий `scripts/daisyworld-luminosity.jl`.

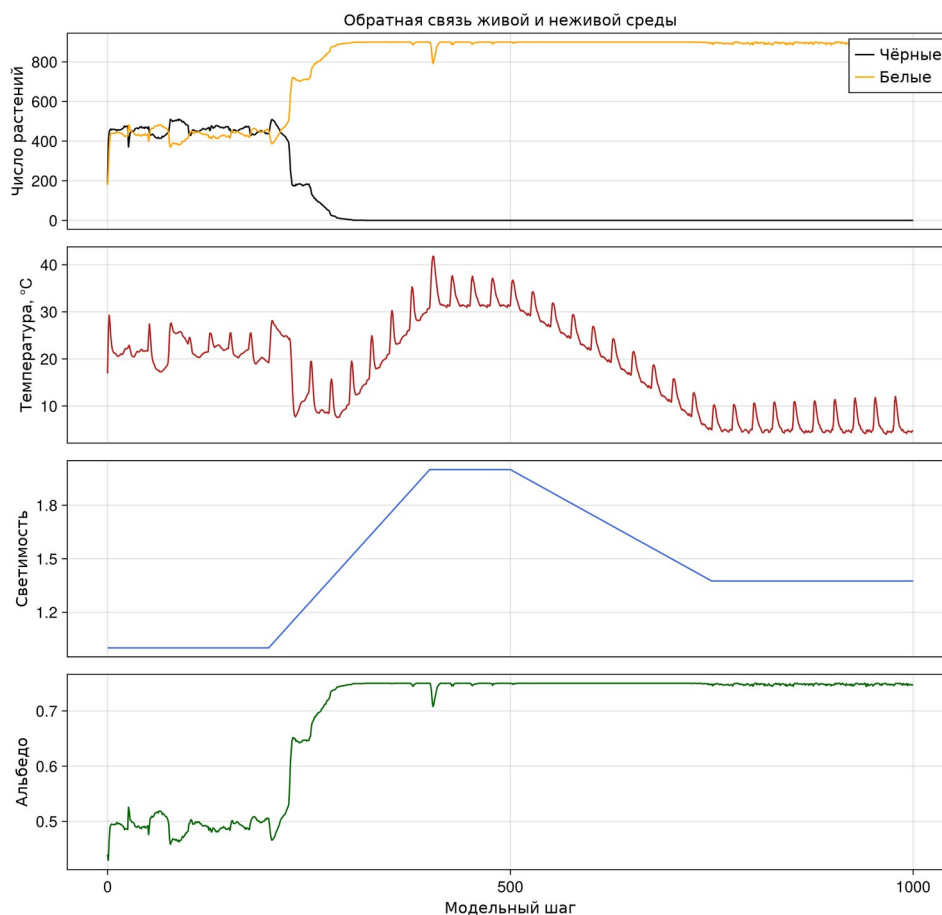


Рисунок 10: Численность, температура, светимость и альbedo

При росте светимости белые маргаритки вытесняют чёрные и повышают среднее альbedo. На шаге 400 все 900 клеток заняты белыми растениями, а средняя температура равна 31,03 °C. После снижения светимости популяция остаётся преимущественно белой,

поэтому к шагу 1000 температура уменьшается до 4,79 °C. Это показывает предел адаптации при быстром внешнем воздействии.

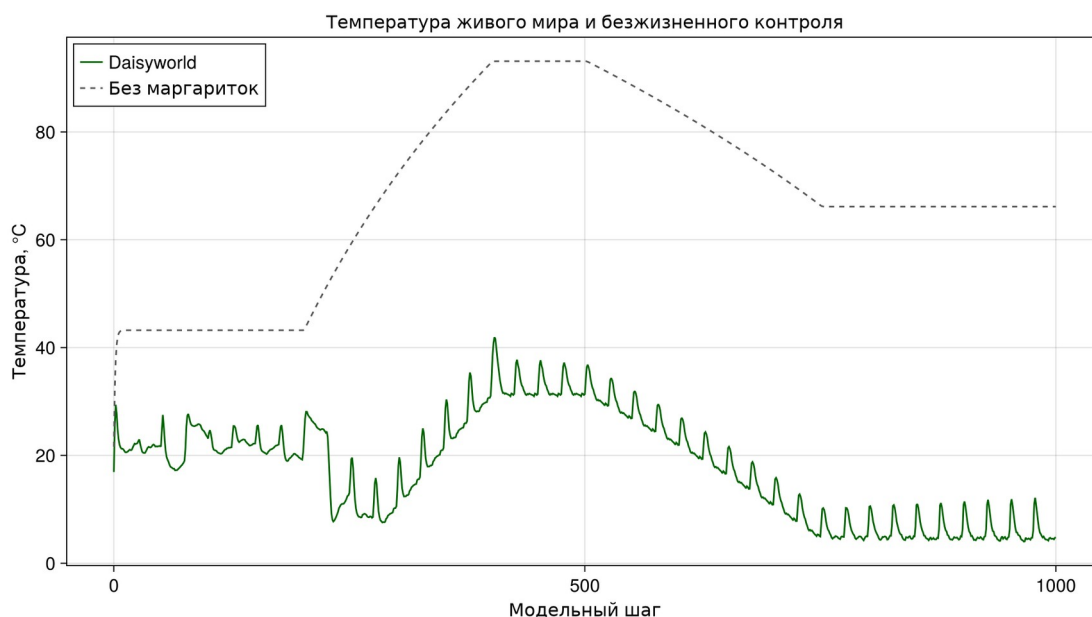


Рисунок 11: Температура Daisyworld и безжизненного контроля

При светимости 2 контроль без маргариток достигает примерно 93,13 °C, тогда как Daisyworld удерживает 31,35 °C. Наибольшее уменьшение отклонения от 22,5 °C составляет 62,32 °C.

4.7 Пространственный параметрический сценарий

4.7.1 Daisyworld: пространственные состояния для набора параметров

Исследуются четыре сочетания максимального возраста 25/40 и начальной доли белых маргариток 0.2/0.8 при остальных параметрах из базового опыта.

4.7.1.1 План эксперимента

```
using DrWatson
@quickactivate "DaisyworldLab"
include(scrdir("experiment_tools.jl"))

rows = NamedTuple[]
final_fig = Figure(size=(1200, 1050))
for (run, parameters) in enumerate(parameter_sets())
    model = create_world(; parameters...)
    for target in (0, 5, 45)
        advance!(model, target - model.tick)
        push!(rows, merge((run=run, max_age=parameters[:max_age],
                           init_white=parameters[:init_white]), observe(model)))
        fig = snapshot_figure(model; title="Опыт $run")
        save_figure(fig, "daisyworld_param",
                    "run-$(run)-step-$(lpad(target, 3, '0')).png")
        display(fig)
    end
end
```



```

end
ax = Axis(final_fig[(run - 1) ÷ 2 + 1, (run - 1) % 2 + 1],
          title="Возраст $(parameters[:max_age]), белые $(parameters[:init_white])",
          aspect=1)
heatmap!(ax, model.temperature; colormap=:thermal, colrange=(-20, 60))
for (breed, colour) in ((:black, :black), (:white, :white))
  points = [Point2f(a.pos...) for a in allagents(model) if a.breed == breed]
  isempty(points) || scatter!(ax, points; color=colour, markersize=5,
                              strokecolor=:gray30, strokewidth=0.3)
end
end

```

4.7.1.2 Сопоставление конечных состояний

```

save_figure(final_fig, "daisyworld__param", "final-state-grid.png";
              report_name="parameter-spatial-comparison.png")
display(final_fig)
summary = DataFrame(rows, summary)
CSV.write(scenario_data("daisyworld__param", "snapshots.csv"), summary)
summary[summary.time .== 45, :]

```

Листинг 10 — сценарий `**scripts/daisyworld__param.jl**`.

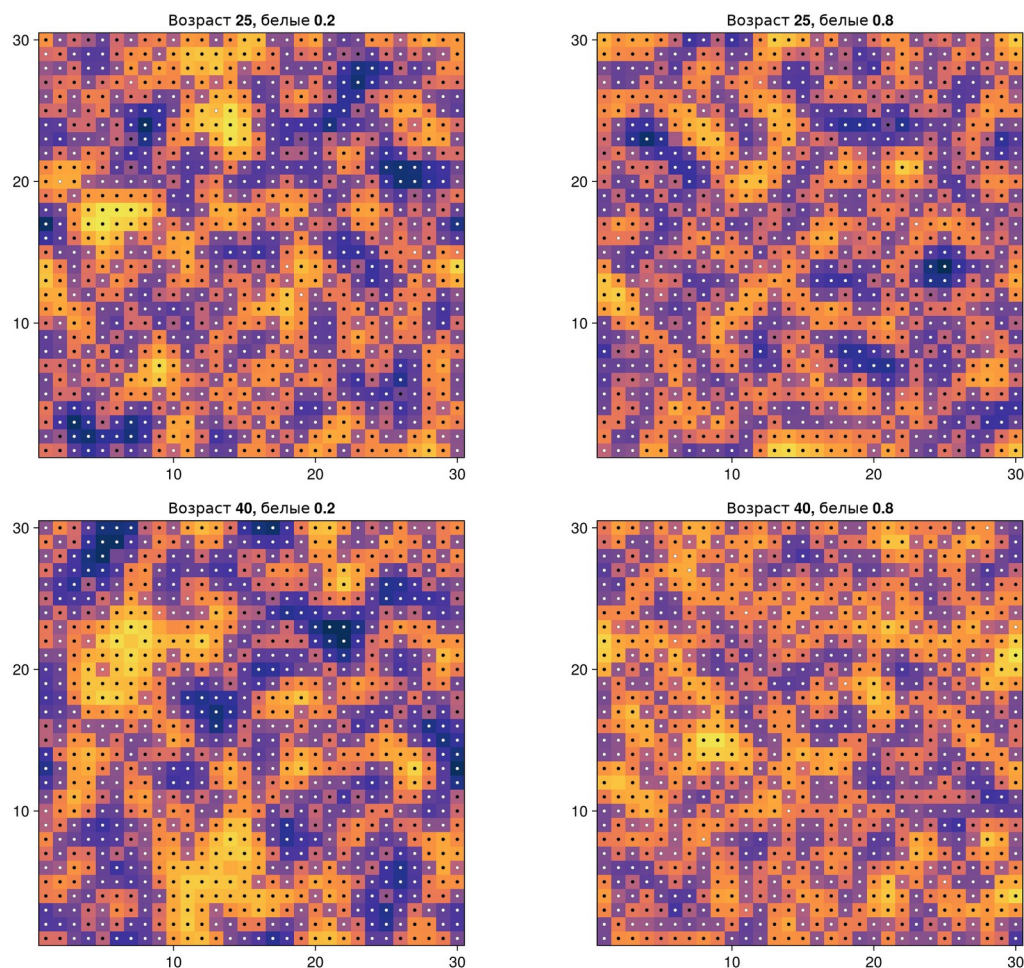


Рисунок 12: Пространственные состояния четырёх опытов

Все сочетания приводят к высокой занятости сетки, но различаются балансом видов. Увеличение возраста до 40 уменьшает частоту освобождения клеток.

4.8 Параметрическая динамика численности

4.8.1 Daisyworld: численность при разных параметрах

Для четырёх сочетаний рассчитываются независимые траектории длиной 1000 шагов.

4.8.1.1 Параметрические прогоны

```
using DrWatson
@quickactivate "DaisyworldLab"
include(scrdir("experiment_tools.jl"))

summary_rows = NamedTuple{()}{()...}()
comparison = Figure(size=(1200, 900))
for (run, parameters) in enumerate(parameter_sets())
    model = create_world(parameters...)
    trajectory = history_frame(model, 1000)
    save_history(trajectory, "daisyworld-count__param";
        filename="trajectory-run-$(run).csv")
    push!(summary_rows, (
        run=run, max_age=parameters[:max_age], init_white=parameters[:init_white],
        final_black=last(trajectory.black), final_white=last(trajectory.white),
        peak_black=maximum(trajectory.black), peak_white=maximum(trajectory.white),
        extinction_step=extinction_time(trajectory),
        mean_regulation_error=mean(trajectory.regulation_error),
    ))
    ax = Axis(comparison[(run - 1) ÷ 2 + 1, (run - 1) % 2 + 1],
        title="Возраст $(parameters[:max_age]), белые $(parameters[:init_white])",
        xlabel="Шаг", ylabel="Число растений")
    lines!(ax, trajectory.time, trajectory.black; color=:black, label="Чёрные")
    lines!(ax, trajectory.time, trajectory.white; color=:orange, label="Белые")
    run == 1 && axislegend(ax; position=:rt)
end
```

4.8.1.2 Итоговые показатели

```
save_figure(comparison, "daisyworld-count__param", "population-grid.png";
    report_name="parameter-population-comparison.png")
display(comparison)
summary = DataFrame(summary_rows)
CSV.write(scenario_data("daisyworld-count__param", "summary.csv"), summary)
summary
```

Листинг 11 — сценарий `**scripts/daisyworld-count__param.jl**`.

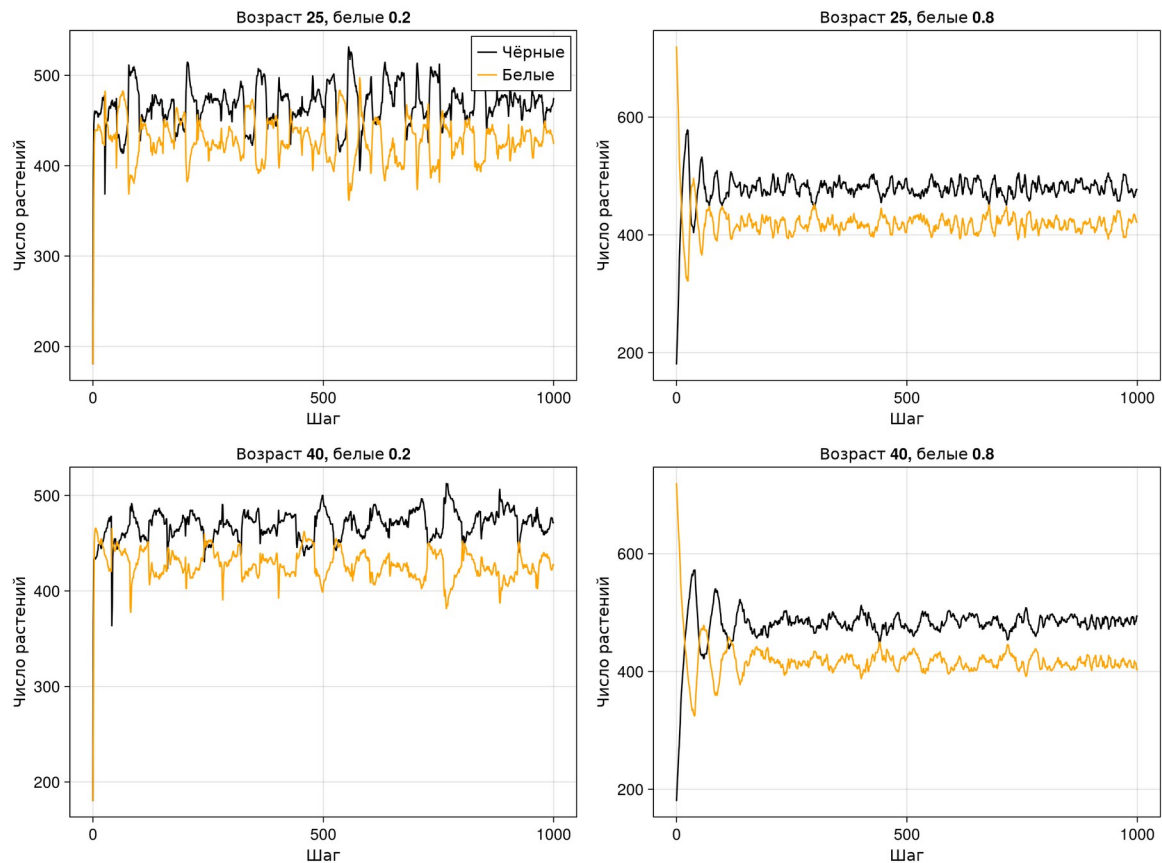


Рисунок 13: Численность для четырёх сочетаний параметров

Ни один опыт не завершился вымиранием. Средняя абсолютная ошибка относительно 22,5 °C находится в диапазоне 1,20–1,68 °C. Наименьшее значение получено при $\text{max_age}=40$ и $\text{init_white}=0,2$.

4.9 Параметрический отклик на светимость

4.9.1 Daisyworld: параметрический отклик на светимость

Для каждого сочетания параметров повторяется полный сценарий ramp на 1000 шагах.

4.9.1.1 Расчёт траекторий

```
using DrWatson
@quickactivate "DaisyworldLab"
include(scrdir("experiment_tools.jl"))

summary_rows = NamedTuple{
comparison, Figure(size=(1200, 900))}
for (run, base_parameters) in enumerate(parameter_sets())
    parameters = copy(base_parameters)
    parameters[:scenario] = :ramp
    model = create_world(parameters...)
    trajectory = history_frame(model, 1000)
    save_history(trajectory, "daisyworld-luminosity__param";
        filename="trajectory-run-$(run).csv")
end
```



```

occupied = trajectory.total .> 0
push!(summary_rows, (
    run=run, max_age=parameters[:max_age], init_white=parameters[:init_white],
    final_total=last(trajectory.total), max_temperature=maximum(trajectory.temperature),
    min_temperature=minimum(trajectory.temperature),
    living_steps=count(occupied), mean_regulation_error=mean(trajectory.regulation_error),
))
ax = Axis(comparison[(run - 1) ÷ 2 + 1, (run - 1) % 2 + 1],
    title="Возраст $(parameters[:max_age]), белые $(parameters[:init_white])",
    xlabel="Шаг", ylabel="Температура, °C")
lines!(ax, trajectory.time, trajectory.temperature; color=:firebrick)
lines!(ax, trajectory.time, 22.5 .+ 10 * (trajectory.luminosity .- 1);
    color=:royalblue, linestyle=:dash)
end

```

4.9.1.2 Сравнение устойчивости

```

save_figure(comparison, "daisyworld-luminosity__param", "temperature-grid.png";
    report_name="parameter-temperature-comparison.png")
display(comparison)
summary = DataFrame(summary_rows)
CSV.write(scenario_data("daisyworld-luminosity__param", "summary.csv"), summary)
summary

```

Листинг 12 — сценарий `**scripts/daisyworld-luminosity__param.jl**`.

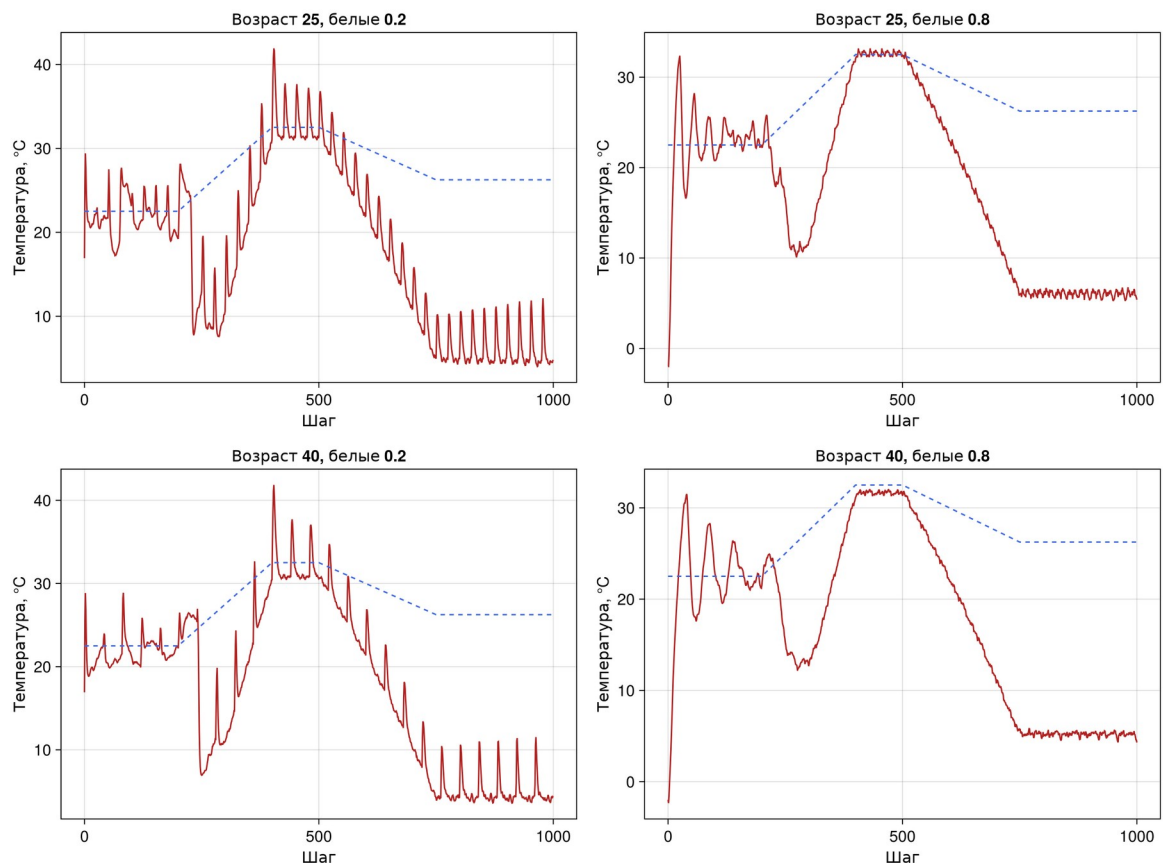


Рисунок 14: Температура в четырёх опытах ramp

Во всех опытах агенты присутствуют в каждом из 1001 сохранённого состояния. Максимальная температура составляет 32,04–41,91 °C. Большая начальная доля белых маргариток ограничивает нагрев, но усиливает переохладение после снижения светимости.

4.10 Литературное программирование и notebooks

Один исходник используется для получения чистого скрипта, QMD и notebook, сохраняя связь объяснения и выполняемого кода [6].

```
using DrWatson
@quickactivate "DaisyworldLab"
using Literate

execute_notebooks = "--execute" in ARGS
sources = filter(!="--execute"), ARGS)

for source in sources
    name = splitext(basename(source))[1]
    Literate.script(source, scriptsdir(name); name=name, credit=false)
    Literate.notebook(
        source, projectdir("notebooks", name);
        name=name, execute=execute_notebooks, credit=false,
    )
    Literate.markdown(
        source, projectdir("markdown", name);
        name=name, flavor=Literate.QuartoFlavor(), credit=false,
    )
    Literate.markdown(
        source, projectdir("markdown", name);
        name=name * "-report", flavor=Literate.QuartoFlavor(), credit=false,
        postprocess=text -> replace(
            replace(text, "{julia}" => "{julia}"),
            r"^(#{1,4}) "m => heading -> "###" * heading,
        ),
    )
end
```

Листинг 13 — генератор производных форматов `scripts/tangle.jl`.

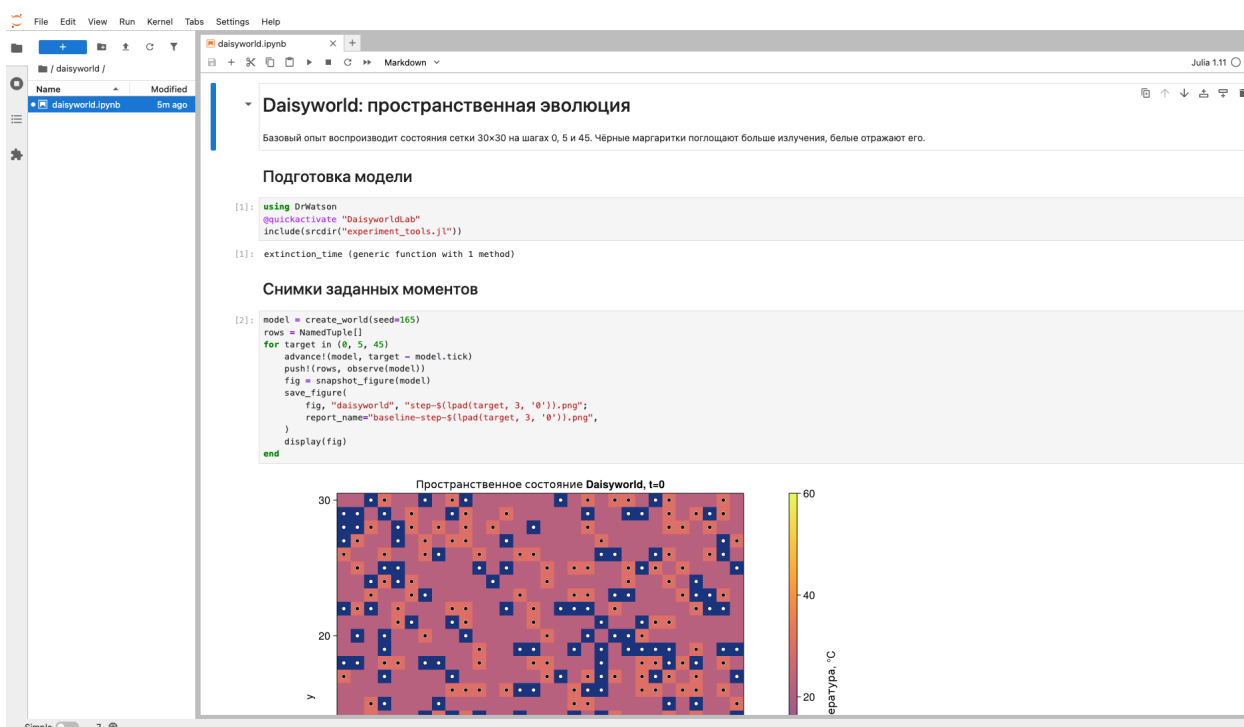


Рисунок 15: Выполненный notebook базового опыта

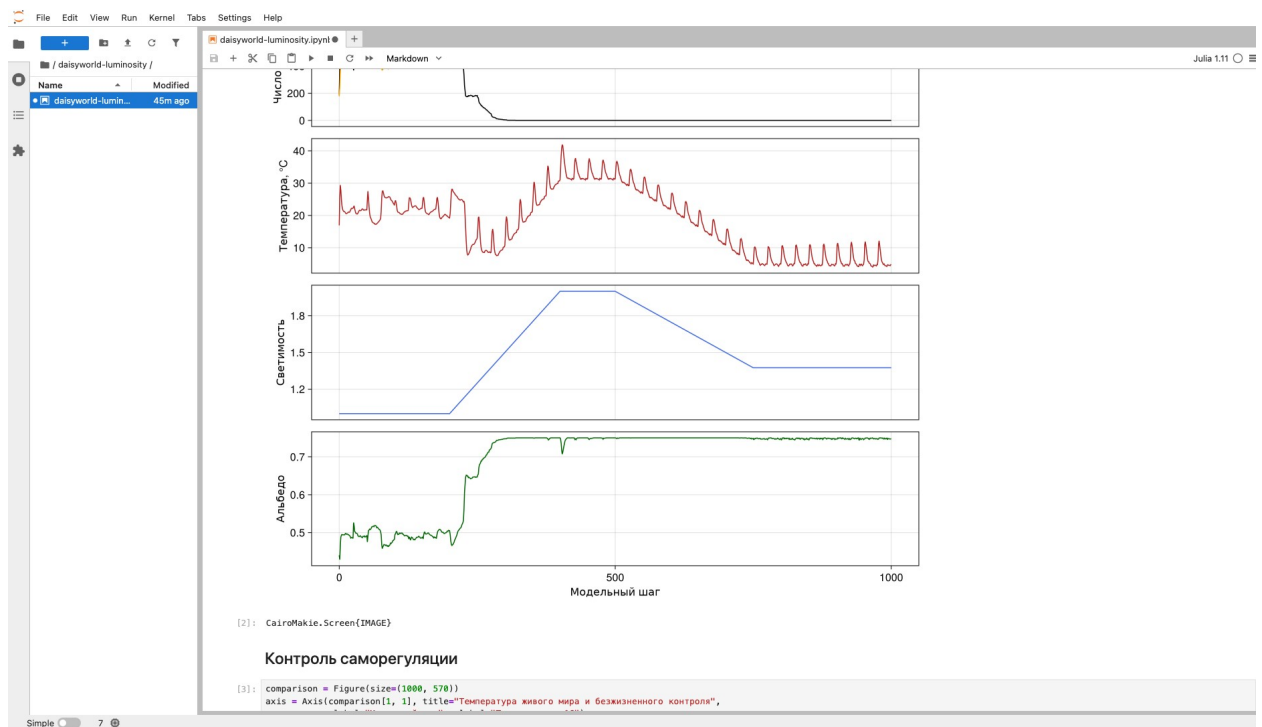


Рисунок 16: Выполненный notebook светового опыта

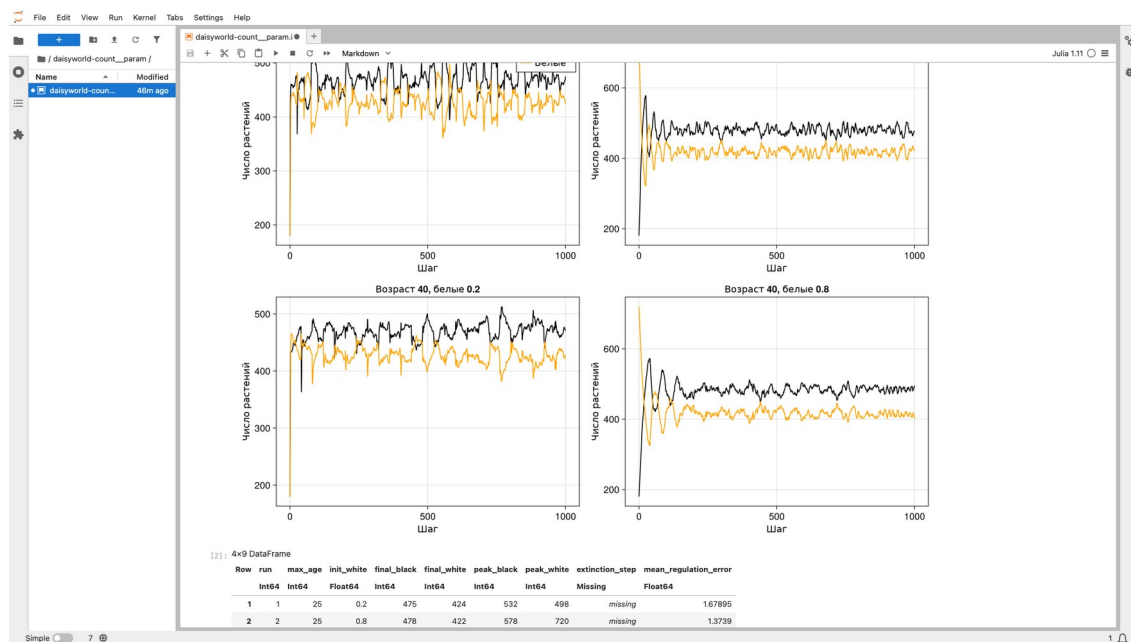


Рисунок 17: Выполненный параметрический notebook

Во всех семи IPYNB заполнены номера выполнения, отсутствуют ошибки и сохранены числовые либо табличные результаты вместе со встроенными изображениями.

4.11 Автоматические проверки

using
using
using

Test
Agents
DataFrames

```

include("../src/daisyworld.jl")
using

.Daisyworld

@testset "Daisyworld" begin
    model = create_world()
    state = observe(model)
    @test state.white == 180
    @test state.black == 180
    @test state.empty == 540
    @test nagents(model) == 360
    @test length(unique(a.pos for a in allagents(model))) == nagents(model)
    @test all(isfinite, model.temperature)
end

@testset "Parameter validation" begin
    @test_throws ArgumentError create_world(griddims=(2, 2))
    @test_throws ArgumentError create_world(max_age=0)
    @test_throws ArgumentError create_world(init_white=0.9, init_black=0.2)
    @test_throws ArgumentError create_world(surface_albedo=1.2)
    @test_throws ArgumentError create_world(diffusion_ratio=-0.1)
    @test_throws ArgumentError create_world(scenario=:unknown)
end

@testset "Reproducibility" begin
    and invariants" begin
        first = create_world(seed=165)
        second = create_world(seed=165)
        advance!(first, 45)
        advance!(second, 45)
        @test observe(first) == observe(second)
        @test first.temperature == second.temperature
        @test first.tick == 45
        @test nagents(first) <= 900
        @test all(a -> a.age < first.max_age, allagents(first))
        @test 0 <= observe(first).albedo <= 1
    end
end

@testset "Empty world" begin
    and solar schedule" begin
        empty = create_world(init_white=0.0, init_black=0.0)
        advance!(empty, 20)
        @test nagents(empty) == 0

        ramp = create_world(scenario=:ramp, init_white=0.0, init_black=0.0)
        advance!(ramp, 200)
        @test ramp.solar_luminosity ≈ 1.0
        advance!(ramp, 200)
        @test ramp.solar_luminosity ≈ 2.0
        advance!(ramp, 100)
        @test ramp.solar_luminosity ≈ 2.0
        advance!(ramp, 250)
        @test ramp.solar_luminosity ≈ 1.375
    end
end

@testset "History schema" begin
    frame = DataFrame(collect_history(create_world(seed=7), 10))
    @test nrow(frame) == 11
    @test frame.time == 0:10
    @test all(frame.total .+ frame.empty .== 900)
    @test all(isfinite, frame.temperature)
end

```

```
end                                     @test      all((0      .<=      frame.albedo)      .&      (frame.albedo      .<=      1))
```

Листинг 14 — полный файл `test/runtests.jl`.

```
rhktrlwmd@Mac % make test
julia --project=. test/runtests.jl
Test Summary: | Pass Total Time
Daisyworld initialization | 6 6 1.1s
Test Summary: | Pass Total Time
Parameter validation | 6 6 0.1s
Test Summary: | Pass Total Time
Reproducibility and invariants | 6 6 0.4s
Test Summary: | Pass Total Time
Empty world and solar schedule | 5 5 0.3s
Test Summary: | Pass Total Time
History schema | 5 5 0.2s
rhktrlwmd@Mac %
```

Рисунок 18: Результаты 28 автоматических проверок

```
rhktrlwmd@Mac % make verify
clean Julia scripts: 7
executed notebooks: 7
Quarto documents: 14
HTML documents: 7
result figures and animation: 22
rhktrlwmd@Mac %
```

Рисунок 19: Сформированные JL, QMD, HTML и IPYNB

Проверены начальные количества, уникальность позиций, ограничения параметров, воспроизводимость, возраст агентов, диапазон альбедо, пустой мир, участки сценария ramp и баланс 900 клеток.

5. Выводы

Реализована пространственная агентная модель Daisyworld и выполнены все семь сценариев главы 3 практикума. Получены базовые состояния, анимация, 1000-шаговые траектории и три параметрических исследования. При постоянной светимости два вида формируют устойчивое заполнение сетки. Сопоставление с безжизненным контролем показывает, как белые растения уменьшают нагрев, но после снижения светимости медленная перестройка состава приводит к переохлаждению.

Из семи литературных исходников сформированы и выполнены семь чистых Jupyter notebooks, семь QMD и семь HTML-документов. Модель и результаты проверены 28 автоматическими тестами.

Список литературы

1. Королькова А. В., Кулябов Д. С. Имитационное моделирование. Практикум. Москва: Российский университет дружбы народов имени Патриса Лумумбы, 2026.
2. Datseris G., Vahdati A. R., DuBois T. C. [Agents.jl: a performant and feature-full agent-based modeling software of minimal code complexity](#) // SIMULATION. 2022. Т. 98, № 11. С. 985–998.
3. Watson A. J., Lovelock J. E. [Biological homeostasis of the global environment: the parable of Daisyworld](#) // Tellus B: Chemical and Physical Meteorology. 1983. Т. 35, № 4. С. 284–289.
4. Wood A. J. и др. [Daisyworld: A review](#) // Reviews of Geophysics. 2008. Т. 46, № 1.
5. Datseris G. и др. [DrWatson: the perfect sidekick for your scientific inquiries](#) // Journal of Open Source Software. 2020. Т. 5, № 54. С. 2673.
6. Knuth D. E. [Literate Programming](#) // The Computer Journal. 1984. Т. 27, № 2. С. 97–111.